Repasos de programación Diseño de entornos digitales web
Laura Sofia Guerrero Ante

FrameWork

Herramienta que proporciona **estructura, reglas y funcionalidades** para facilitar el desarrollo de aplicaciones.
NestJS → framework para backend

React → framework/librería para frontend

NestJS

NestJS es un framework de **Node.js con TypeScript** para crear aplicaciones **backend y APIs** de forma estructurada.

Organiza el código en:

- **Controllers** → reciben las peticiones

- **Services** → lógica del programa
- todo lo que la entidad puede hacer

- **Modules** → organizan la aplicación

API

Una **API (Application Programming Interface)** permite que **dos aplicaciones se comuniquen entre sí**.

Ejemplo:

Frontend → API → Backend → Base de datos

Las APIs usan métodos HTTP como:

Método	Acción
GET	obtener datos
POST	crear datos

PUT	actualizar datos
DELETE	eliminar datos
	Patch tasks (actualización parcial)

Servidor

Un **servidor** es un programa que **recibe peticiones y devuelve respuestas**.

Ejemplo:

Usuario → Servidor → Base de datos → Servidor → Usuario

En tu proyecto el servidor es **NestJS corriendo en Node.js**

Diferencias entre node node.js y nestJS:

1. Node.js es un entorno para ejecutar JS mientras que NestJS es un framework construido sobre node.

2. Node.js es la base del backend mientras que NestJS es la estructura para organizar el backend

Node.js = motor

NestJS = estructura para construir la app

PostgreSQL

PostgreSQL es un **sistema de base de datos relacional** donde se almacenan los datos.

Usa **SQL** para manejar información como:

- usuarios

- productos
- pedidos

TypeORM

TypeORM es una herramienta que conecta el **backend con la base de datos**.

Permite usar **clases de TypeScript como tablas de base de datos**.

Ejemplo:

```
@Entity()
class User {
 @Column()
 name: string
}
```

Eso crea una tabla en la base de datos.

Docker

Docker es una herramienta que permite ejecutar aplicaciones en **contenedores aislados**.

Sirve para que todos los desarrolladores usen **el mismo entorno**.

Ejemplo en tu proyecto:

NestJS → TypeORM → PostgreSQL (Docker container)

Que son las HTTP?

Es el protocolo que permite la comunicación entre el cliente y el servidor de la web que son los más usados como el **get** el **post** el **put** y el **delete**

resumen :3

NestJS es un framework de Node.js que permite crear APIs backend estructuradas.
Estas APIs reciben peticiones HTTP y realizan operaciones CRUD sobre una base de datos, usando herramientas como TypeORM para conectarse a sistemas de

almacenamiento como PostgreSQL, que en muchos casos se ejecutan en contenedores Docker.

Dia 2

Estructura de NestJS

Module

Controller

Service

Entity

DTO

CRUD con TypeORM

find()

findOneBy()

save()

update()

delete()

Decoradores importantes

@Controller()

@Get()

@Post()

@Put()

@Delete()

@InjectRepository(

Mucho código **se repite**.

Por ejemplo en controller casi siempre verás:

@Get()

@Post()

@Put(':id')

@Delete(':id')

y en service:

findAll()

findOne()

create()

update()

remove()

Que es nest-cli.json en visual estudio code

es para configurar el framework

Que es una Entidad (Entity)

Una entidad es una clase que representa una tabla en la base de datos.

📌 **Palabras clave:**
`tabla` • `datos` • `database`

Ejemplo:

@Entity()

class User {

name: string

}

Que es un Módulo (Module)

Un módulo organiza y agrupa partes de la aplicación como controllers y services.

📌 **Palabras clave:**
`organiza` • `estructura` • `componentes`

Ejemplo:

@Module({

controllers: [UsersController],

providers: [UsersService],

})

Códigos en terminal importantes

si pones nest en la terminal te aparecerá tablas con códigos y su uso despues eliges en la tabla el que quieres usar pero antes debes poner

nest generate … y en los puntos pones lo que desees

ejemplo con module

nest generate module tasks , el tasks singifca tareas por que de ese trata mi trabajo y asi se llama la carpeta que se mostrará en visual

ejemplo con controller

nest generate controller tasks --no-spec , este lo que hace es crear los archivos con solo controlador si queremos el de test tenemos que quitarle el –no-spec

npm install uuid

Esto nos ayuda en la terminal a que va a instalar un modulo extra y lo importamos en la carpeta donde deseamos y esto va a generar id unicos

Que son los DTos ? (data transfer object)

Es una clase que define **los datos que recibes**

EJEMPLO

export class CreateProductDto {

name: string;

price: number;

También sirve para hacer validaciones como por ejemplo

(class-validator)

Sirven para validar datos antes de llegar al service

Ejemplo:

```
import { IsString, IsNumber, IsNotEmpty } from 'class-validator';


export class CreateProductDto {
 @IsString()
 name: string;


 @IsNumber()
 price: number;
}
```

en resumen

DTO = valida datos (tipo)

Service = valida reglas (lógica)

Dia 3 Crear y listar

los servicios sirven para crear métodos que se pueden reutilizar

Memorizar esto:

Controller → recibe
Service → lógica
Repository → DB
Las validaciones y reglas SIEMPRE van en el service
El controller solo recibe y llama al service

CRUD en español

```
findAll → traer todos
findOne → traer uno
create → crear
update → actualizar
remove → eliminar
```

Relaciónarlo con HTTP

```
GET → findAll
GET /:id → findOne
POST → create
PUT → update
DELETE → remove
```

Entender el flujo

GET /users

```
→ controller
→ service
→ repository
→ database
```

@Body

significa que el usuario manda el mensaje en forma json

```
@Body = datos enviados en la petición
GET → no usa body
POST / PUT → sí usan body
DELETE → normalmente no usa body
es decir
si creas o actualizas → @Body
si lees → no @Body
```

npm install uuid

Esto nos ayuda en la terminal a que va a instalar un modulo extra y lo importamos en la carpeta donde deseamos y esto va a generar id unicos

DIa 4 Actualizar y eliminar

@Param

Este param

Object.assign esto nos ayuda en postman ya que si queremos actualizar algo podemos poner const a el original y const b el actualizado y si usamos este comando los va a juntar

DIA 4 VALIDACIONES

```
$ npm i --save class-validator class-transformer
```

```
es un comando para terminal para usarlo en la carpeta de Dto
de nuestro archivo y tambien importarlo en la Dto como
class-validator y con decoradores usando IS adentro de las
llaves que tiene
```

Main

en el archivo main debemos llevar este siguiente código

```
app.useGlobalPipes(new ValidationPipe());
```

¿Qué es BadRequestException?

Es un **error que envía NestJS cuando algo está mal en la petición**

BadRequestException = error cuando los datos están mal

NotFoundException = error cuando no existe algo

Inicializacion para el proyecto

en la terminal escribimos los siguientes

Entrar al proyecto

git clone
cd proyecto
npm install
npm run start:dev

SI HAY DOCKER

docker compose up

Instalar base de datos (TypeORM + Postgres)

npm install @nestjs/typeorm typeorm pg

Crear módulo, service y controller

Para cliente:

nest g module clients
nest g service clients
nest g controller clients

PARCIAL IMPORTANTE

1. DTOs + class-validator (10%)

✔ Qué esperan:

- DTOs separados
- Validaciones con decorators

¿QUÉ HACE ESTO?

✔ `@IsString()` → debe ser texto
✔ `@IsNumber()` → debe ser número
✔ `@Min(0)` → mínimo 0

```typescript
import { IsString, IsNumber, Min } from 'class-validator';

export class CreateClienteDto {
  @IsString()
  nombre: string;

  @IsNumber()
  @Min(0)
  score_crediticio: number;
}
```

2. Controller + Service “El corazón”

Regla clave:

- Controller = recibe petición
- Service = hace TODO

  Ejeemploo

```
// controller
@Post()
create(@Body() dto: CreateDto) {
 return this.service.create(dto);
}
// service
create(dto: CreateDto) {
 // lógica aquí
}
```

3. Lógica de negocio → “Lo importante”

Ejemplo real de el pre parcial nest 1 :

```
if (cliente.score_crediticio <= 700) {
 throw new BadRequestException('Score bajo');
}
```

Eso es lo que más vale ya que es algo molesto

actualizar → `Object.assign()`

🛡️ 4. Seguridad (15%) → “Proteger rutas”

RESUMEN SIMPLE (para que lo entiendas antes de verlo)

Seguridad en Nest funciona así:

1. Usuario hace login
2. El backend genera un **token (JWT)**
3. El usuario envía ese token en cada petición
4. Un **Guard** valida el token
5. Si es válido → entra
6. Si no → ❌ error

Los guards controlan el acceso a rutas

```
@UseGuards(AuthGuard)
@Get()
findAll() {}
```

🚨 5. Excepciones (10%)

Solo validar errores

```
if (!user) {

  throw new NotFoundException('No existe');

}

throw new NotFoundException('No existe');
throw new BadRequestException('Error');
```

5. Service (lógica – LO MÁS IMPORTANTE

```
create(dto: CreateUserDto) {
  return this.users.push(dto);
}
```

Aquí haces:

- reglas
- validaciones extra

- lógica

6. Controller (endpoints)

```
@Post()
create(@Body() dto: CreateUserDto) {
  return this.userService.create(dto);
}
@Post()
create(@Body() dto: CreateUserDto) {
  return this.userService.create(dto);
}
```

conecta HTTP con el service